

Functional Languages

Short Answer Questions

01. List the four key principles around which functional programming is built

Pure functions, immutability, first-class/higher-order functions, and recursion

02. Define "immutability" in the context of functional programming.

It means that data, once created, cannot be changed; instead of modifying structures, new ones are created

03. State one major advantage of using functional programming regarding concurrency

Code with no shared mutable state is inherently thread-safe, simplifying parallel application development

04. List three examples of languages that support functional programming.

Haskell, Lisp, and Scala

05. Define a higher-order function.

A function that takes other functions as arguments, returns them as results, or assigns them to variables

06. Give two examples of common higher-order functions used for collection processing.

Map, filter, and reduce are common examples

07. Define "referential transparency."

It is the property where a function call can be replaced by its resulting value without changing the program's meaning

08. What is a "pure function"?

A function whose output depends only on its inputs and produces no side effects

09. What is "lazy evaluation"?

It is an evaluation strategy where expressions are not evaluated until their results are actually needed

10. Identify a pure functional language that uses lazy evaluation by default.

Haskell

11. What is "Normal Order" evaluation?

It is a strategy that reduces the outermost expressions first and guarantees a result if one exists

12. What is "Applicative Order" evaluation?

It is a strategy where arguments are reduced (evaluated) first before the function body is applied

13. List the three core syntactic components of Lambda Calculus.

Variables, Function Abstraction, and Application

14. What does the symbol (lambda) represent in this system?

It indicates the start of a function abstraction (definition)

15. Differentiate between a bound variable and a free variable.

A bound variable appears inside a lambda abstraction, while a free variable is not bound by any lambda

Long answer questions

01. Define the Functional Programming paradigm. Discuss its origins, core principles, and the role of "pure functions" within this paradigm. Explain how these concepts influence modern languages and their relevance to compiler optimization.

- **Definition and Origin:** Functional programming is a paradigm where computation is treated as the evaluation of a series of mathematical functions rather than the execution of commands or state changes. Its roots lie in mathematics, and it is language-independent, drawing heavily from the formal system of the Lambda Calculus introduced by Alonzo Church in the 1930s.
- **Core Principles:** The paradigm is built around several key principles:
 1. **Pure Functions:** Functions whose output depends only on their inputs.
 2. **Immutability:** Data cannot be changed once created; new structures are created instead of modifying existing ones.
 3. **First-Class Functions:** Functions are treated as values.
 4. **Recursion:** Used for iteration instead of loops,
- **Role in Functional Programming:** Pure functions are central to the model because they eliminate side effects (such as modifying external variables or I/O). This ensures that calling a function with the same arguments always produces the same result, a property known as referential transparency.
- **Influence in Modern Languages:** While languages like Haskell are purely functional, modern multi-paradigm languages like Python, Java, and JavaScript have incorporated functional features. For instance, Python utilizes lambda expressions and map/filter functions, and Java (since version 8) supports lambdas
- **Relevance in Compilers:** The absence of side effects and the property of referential transparency allow compilers to perform optimizations more aggressively. Because the execution order of pure functions does not alter the program state, compilers can reorder instructions or run tasks in parallel without data race concerns

02. Explain the concept of Lambda Calculus. Detail its syntax, the process of Beta-reduction, and its role as the theoretical foundation for functional languages. How does it relate to the "Turing Complete" status of a language?

- **Definition & Origin:** A formal mathematical system introduced by Alonzo Church in the 1930s to model computation.
- **Core Principles:** Built on variables, function abstraction ($\lambda x.M$), and application (MN). Beta-reduction is the process of applying a function to an argument by substitution,
- **Role in FP:** It serves as the smallest universal programming language and the basis for Lisp, Haskell, and ML,
- **Influence:** Concepts like anonymous functions in Python or JavaScript are direct implementations of lambda abstractions.
- **Compiler Relevance:** Being Turing complete means it can express any computable function, making it a powerful model for language semantics and type systems in compiler design

03. Compare "Strict" (Applicative Order) and "Lazy" (Normal Order) evaluation. Discuss their origins in Lambda Calculus, their benefits regarding data structures, and the trade-offs involved in compiler implementation.

- **Definition & Origin:** Evaluation strategies from Lambda Calculus; Strict reduces arguments before application, Lazy reduces the outermost expression first.
- **Core Principles:** Strict computes everything immediately; Lazy computes values only when strictly needed.
- **Role in FP:** Lazy evaluation allows for the creation of infinite lists or data streams.
- **Influence:** Haskell uses lazy evaluation by default; Python uses it for generators.
- **Compiler Relevance:** Normal order (Lazy) guarantees a result if one exists (avoiding some infinite loops), but Strict order is generally more efficient to implement in compilers and mirrors standard hardware execution

04. Explain the role of "Recursion" in functional languages. How does it replace traditional iteration? Discuss its theoretical basis in the Y-Combinator and how compilers optimize recursive functions.

- Definition & Origin: A function calling itself to solve a problem; theoretically based on fixed-point combinators like the Y-Combinator in Lambda Calculus,
- Core Principles: Breaking problems down without mutable loop counters.
- Role in FP: The primary mechanism for iteration since traditional loops require mutable state.
- Influence: Fundamental to all functional languages and widely used in algorithms for tree traversal.
- Compiler Relevance: Compilers must implement Tail Call Optimization (TCO) to reuse stack frames, converting recursion into iteration at the machine level to avoid stack overflow.

05. What does it mean for functions to be "First-Class Citizens"? Explain the origin of this concept, its practical applications in functional programming, and how it impacts the "Type System" of a language.

- Definition & Origin: Functions are treated as values (passed, returned, assigned); a direct concept from Lambda Calculus,.
- Core Principles: Code is treated as data.
- Role in FP: Enables the creation of higher-order functions and powerful abstractions.
- Influence: Adopted by imperative languages like C# and Java (lambdas) to support functional patterns.
- Compiler Relevance: Requires Typed Lambda Calculus foundations in the type system to validate function signatures (e.g., checking if a parameter is a function `Int -> Int`)

Structured Question

Explain the core paradigms of Functional Programming.

a) Define the Functional Programming paradigm. (3 marks)

b) Explain the concepts of "Pure Functions" and "Immutability". (4 marks)

c) How is iteration handled in this paradigm without traditional loops? (4 marks)

d) List three examples of languages that support this paradigm. (4 marks)

a) Functional programming is a paradigm where computation is treated as the evaluation of mathematical functions rather than the execution of commands or state changes.

b) **Pure Functions** are functions whose output depends only on their inputs and produce no side effects. **Immutability** means data cannot be changed once created; new structures are created instead of modifying existing ones.

c) Iteration is achieved through **recursion**, where a function calls itself until a base condition is met, rather than using mutable loop counters (like for or while).

d) Haskell (pure), Lisp, and Scala (supports FP features)